

APPLIED ARTIFICIAL NEURAL NETWORK



Chapter 3

Typical neural networks

3.1 Introduction

3.2 RBF Networks

3.3 Recurrent neural networks

3.4 Convolution neural networks

REFERENCES

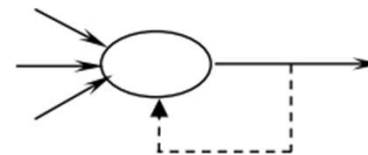
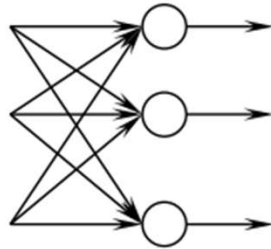
1. Charu C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Springer, 2018
2. Michael Nielsen, *Neural Networks and Deep Learning*, Google Book, 2019
3. Ahmed Gad, Fatima Jarmouni, *Introduction to Deep Learning and Neural Networks with Python*, Elsevier, 2020



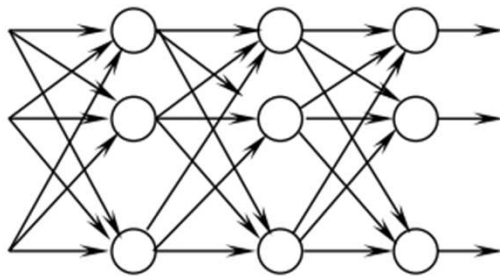
3.1 Introduction

3.1 Introduction

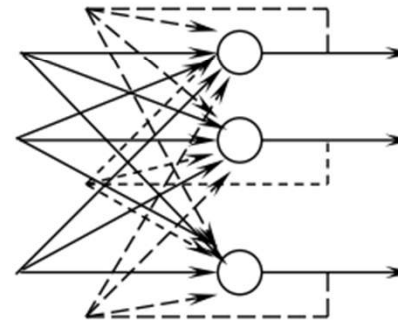
Feed-forward
network



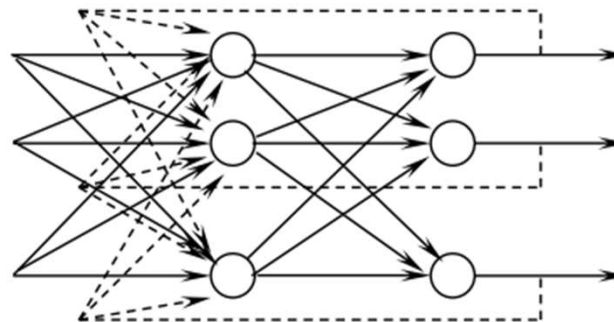
A neuron with
feedback to itself



Feed-forward
network with
multiple layers



Recurrent network
with single layer



Recurrent network
with multiple layers



3.2 RBF Networks

3.2 RBF Networks

- Radial basis function (RBF) networks is a different architecture from a feed-forward network.
- An RBF network typically uses only an input layer, a single hidden layer (with a special type of function defined by RBF), and an output layer.

3.2.1 Network Architecture

❖ RBF Network Architecture:

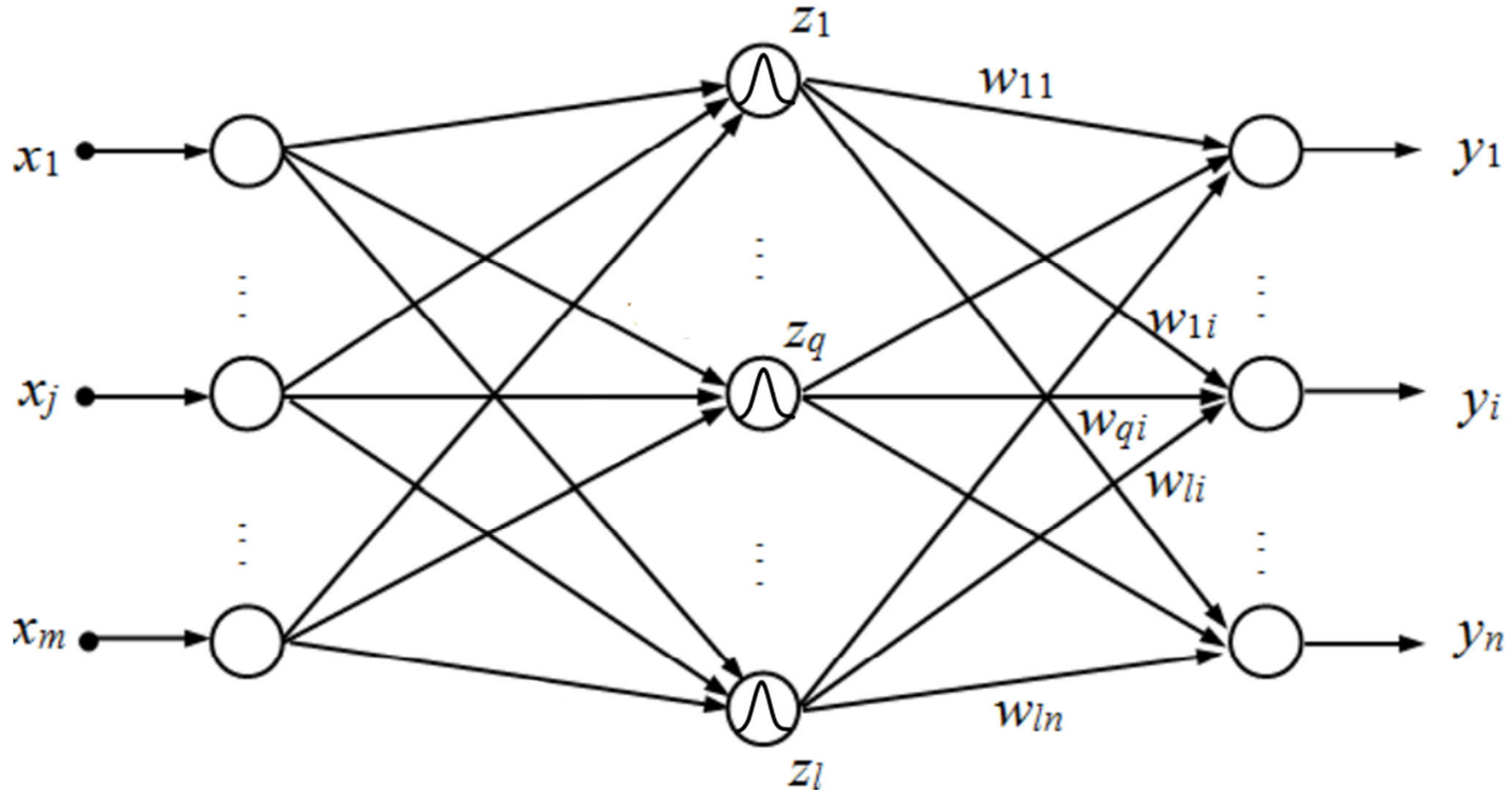


Fig. 3.1: RBF Network Architecture

3.2.1 Network Architecture

The RBF network architecture is described as shown in Fig. 3.1:

❖ **Input layer:** $\mathbf{x} = [x_1, x_2, \dots, x_m]^T \in \mathbb{R}^m$

- The input layer simply transmits from the input features to the hidden layers.
- The number of input units is exactly equal to the dimensionality m of the data.
- There is not computation is performed in the input layers.
- The input units are fully connected to the hidden units and carry their input forward.

3.2.1 Network Architecture

The RBF network architecture is described as shown in Fig. 3.1:

❖ Hidden layer:

- The activation function : Radial basis function

$$\varphi_q(x) = \exp\left(-\frac{\|x - \mu_q\|^2}{2\sigma_q^2}\right) \quad \forall q \in (1, 2, \dots, l)$$

where $\mu_q \in \mathbb{R}^m$ is called center of RBF q^{th}

$\sigma_q \in \mathbb{R}$ is called spread of RBF q^{th}

$\varphi_q \in \mathbb{R}$ is called radial basis function q^{th}

3.2.1 Network Architecture

The RBF network architecture is described as shown in Fig. 3.1:

❖ Hidden layer:

▪ **Note :**

- The total number of hidden units is denoted by l . It is larger than the input dimensionality m , but smaller than the number of training points. (*This principle is based on Cover's theorem on separability of patterns*).
- $\varphi_q(x)$ depends on the distance of the input x from the center μ_q

3.2.1 Network Architecture

The RBF network architecture is described as shown in Fig. 3.1:

❖ Out layer:

- The activation function : Linear function

$$a(f) = f \quad \forall i \in (1, 2, \dots, n)$$

- The threshold level of neurons is zero.
- Output computation y :

$$y = \sum_{i=1}^n y_i$$

3.2.1 Network Architecture

The RBF network architecture is described as shown in Fig. 3.1:

❖ Out layer:

- Output computation y_i :

$$y_i = \sum_{q=1}^l w_{iq} \varphi_q(x) = \sum_{q=1}^l w_{iq} \exp\left(-\frac{\|x - \mu_q\|^2}{2\sigma_q^2}\right)$$

where

$w_i \in \mathbb{R}^l$ are called weights of output i^{th}

3.2.1 Network Architecture

❖ Applications:

The RBF networks can be used classification and function approximation problems.

3.2.2 Separability of patterns

A binary partition, also called dichotomy, (C_1, C_2) of the training set C is separable if there is a vector w of dimension l such that:

$$w^T \varphi(x) > 0 \quad x \in C_1$$

$$w^T \varphi(x) < 0 \quad x \in C_2$$

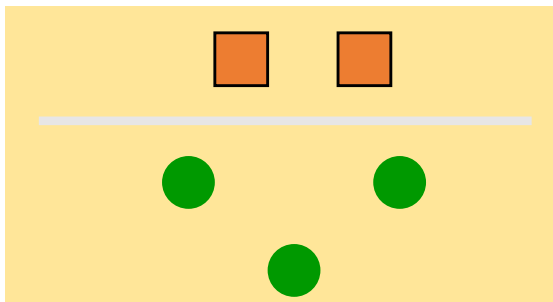
3.2.2 Separability of patterns

❖ Examples of ϕ -separability:

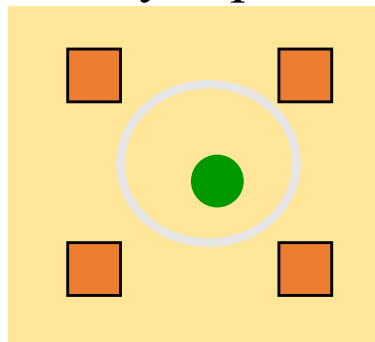
Separating surface: all x such that $w^T \phi(x) = 0$

Examples of separable partitions (C1,C2):

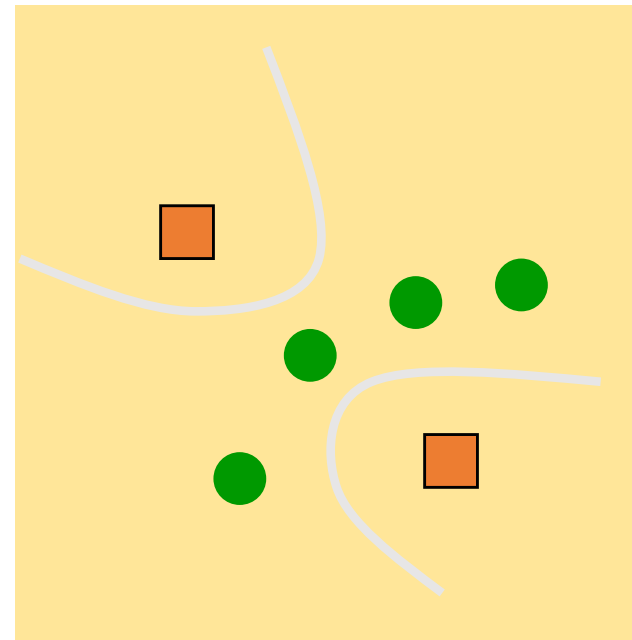
Linearly separable:



Spherically separable:

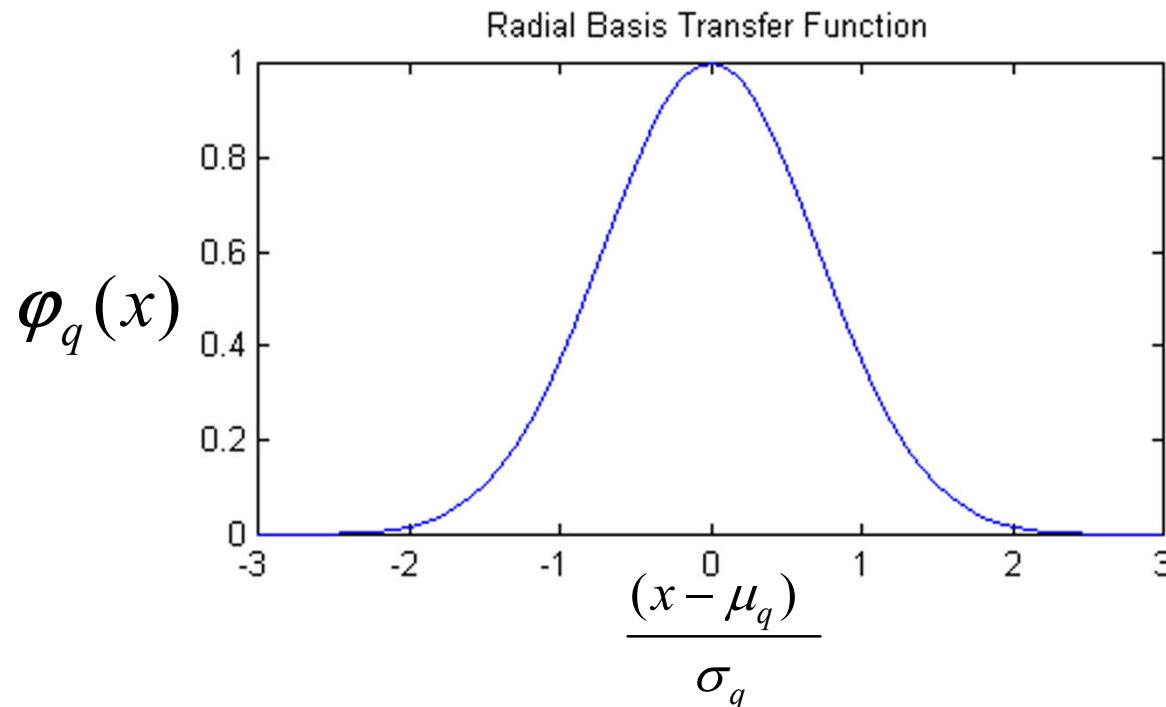


Quadratically separable:



3.2.3 Radial basis function

❖ Radial Basis Transfer Function:



- A hidden neuron is more sensitive to data points near its center. This sensitivity may be tuned by adjusting the spread σ .
- Larger spread $\Rightarrow$ less sensitivity

3.2.3 Radial basis function

❖ Types of φ :

- Multiquadrics:

$$\varphi(r) = (r^2 + c^2)^{1/2} \text{ where } c > 0$$

- Inverse multiquadrics: $r = \|x - \mu\|$

$$\varphi(r) = \frac{1}{(r^2 + c^2)^{1/2}} \text{ where } c > 0$$

- Gaussian functions:

$$\varphi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right) \text{ where } \sigma > 0$$

3.2.4 Learning Algorithm

Parameters to be learnt are:

- Centers
- Spreads
- Weights

Learning Algorithm 1:

- Centers are selected at random: Center locations are chosen randomly from the training set.
- Spreads are chosen by normalization:

$$\sigma = \frac{\text{Maximum distance between any 2 centers}}{\sqrt{\text{number of centers}}} = \frac{d_{\max}}{\sqrt{l}}$$

$$\varphi_i(x) = \exp\left(-\frac{l}{d_{\max}^2} \|x - \mu_i\|^2\right)$$
$$i \in [1, \dots, l]$$

Learning Algorithm 2:

Weights of output are updated by LMS algorithm:

$$w_i = \left[\sum_{k=1}^K z(k) z^T(k) \right]^{-1} \left[\sum_{k=1}^K z(k) d_i(k) \right]$$

where $z(k) = [z_1(k) \quad z_2(k) \quad \cdots \quad z_l(k)]^T$

$$z_q(k) = \exp\left(-\frac{\|x(k) - \mu_q\|^2}{2\sigma_q^2}\right)$$

K is number of training data set

Learning Algorithm 2: the output of RBF

Step 1: - Choose η , $E_{\max}$;

- Initialize random weights w_{iq} ($i = \overline{1, n}; q = \overline{1, l}$)
; $E=0$.

Step 2: Calculate the output of the neural network

$$y_i = \sum_{q=1}^l w_{iq} \varphi_q(x) = \sum_{q=1}^l w_{iq} \exp\left(-\frac{\|x - \mu_q\|^2}{2\sigma_q^2}\right)$$

Step 3: Update weights of the output layer

$$w_{iq}(k+1) = w_{iq}(k) + \eta(d_i(k) - y_i(k)) \cdot z_q(k); \quad i = \overline{1, n}; \quad q = \overline{1, l};$$
$$z_q(k) = \exp\left(-\frac{\|x(k) - \mu_q\|^2}{2\sigma_q^2}\right)$$

Step 4: Calculate error:

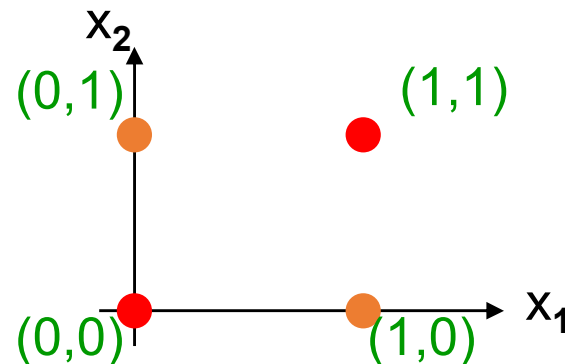
$$E = E + \frac{1}{2} \sum_{i=1}^n (d_i(k) - y_i(k))^2$$

Step 5: If $k < \text{data_length}$ then $k = k + 1$, go to step 2. If $k = \text{data_length}$ then go to step 6.

Step 6: If $E < E_{\max}$ then end of training cycle. If $E \geq E_{\max}$ then lets $E = 0$, $k = 0$ and go to Step 2.

Example: the XOR problem

- Input space:



- Output space:



- Construct an RBF pattern classifier such that:
 $(0,0)$ and $(1,1)$ are mapped to 0, class C1
 $(1,0)$ and $(0,1)$ are mapped to 1, class C2

Example: the XOR problem

- In the feature (hidden) space:
 - Center locations are chosen from the training set:

$$\mu_1 = [1, 1]^T; \mu_2 = [0, 0]^T$$

- Spreads are chosen by normalization:

$$\sigma = \frac{d_{\max}}{\sqrt{l}} = \frac{\sqrt{2}}{\sqrt{2}} = 1$$

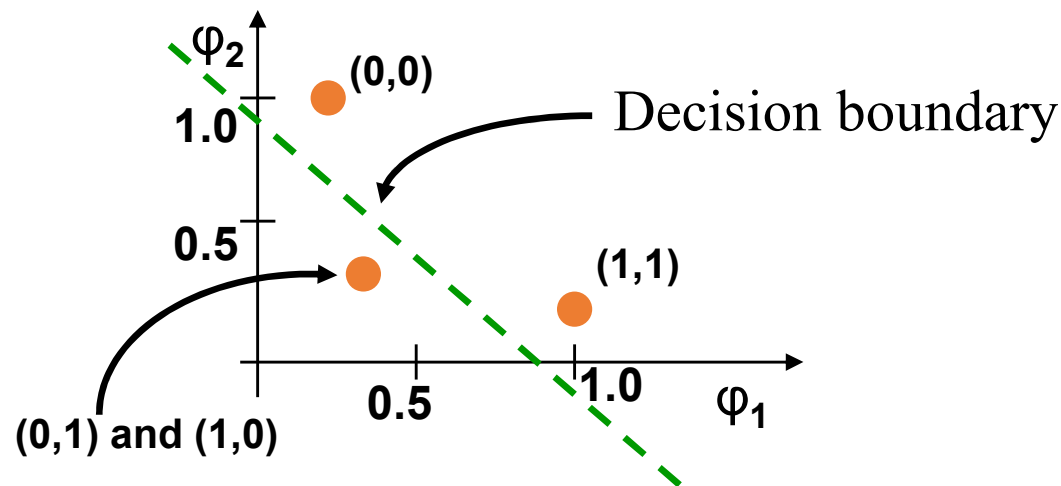
- Radial basis functions:

$$\varphi_1(x_1, x_2) = e^{-\|x - \mu_1\|^2}$$

$$\varphi_2(x_1, x_2) = e^{-\|x - \mu_2\|^2}$$

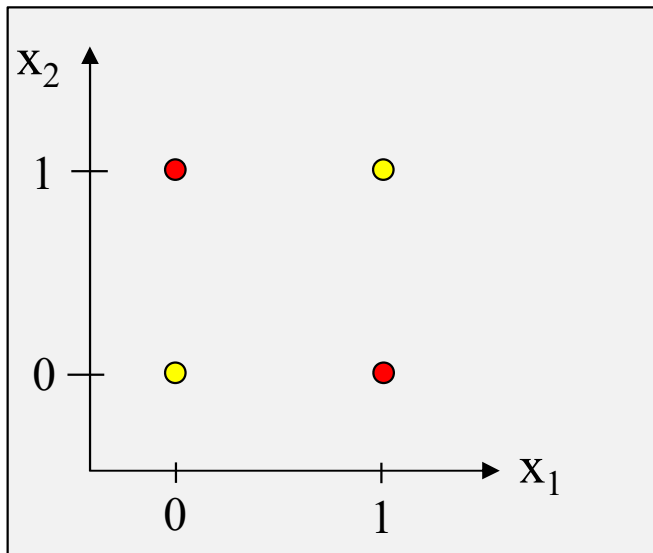
Example: the XOR problem

- In the feature (hidden) space:



When mapped into the feature space $\langle \phi_1, \phi_2 \rangle$, C1 and C2 become *linearly separable*. So a linear classifier with $\phi_1(x)$ and $\phi_2(x)$ as inputs can be used to solve the XOR problem.

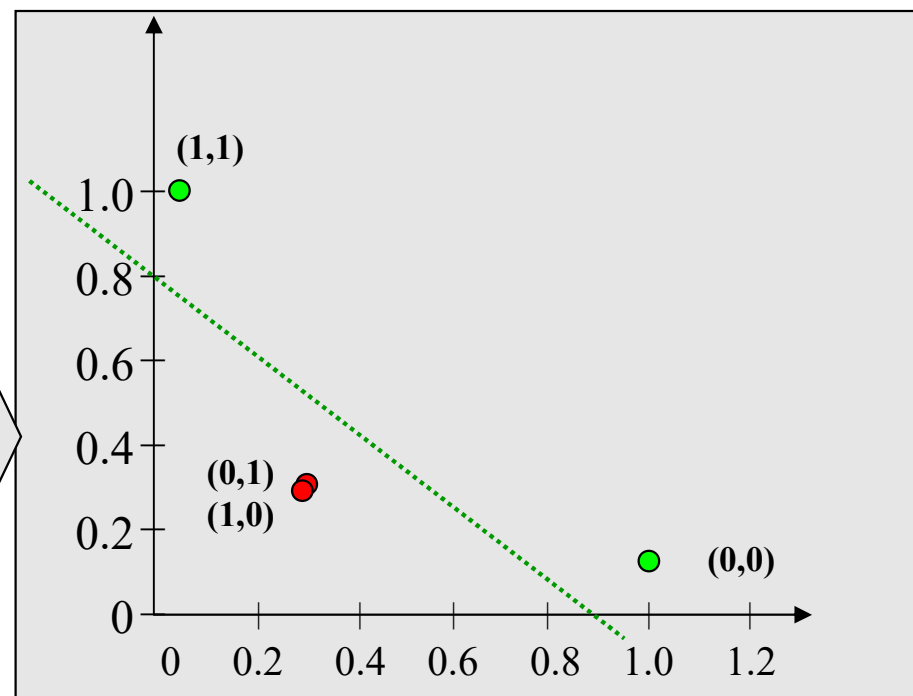
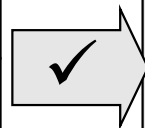
Example: the XOR problem



Consider the nonlinear functions to map the input vector $\mathbf{x}$ to the φ_1 - φ_2 space:

$$\mathbf{x} = [x_1 \ x_2]$$
$$\varphi_1(\mathbf{x}) = e^{-\|\mathbf{x} - \mu_1\|^2} \quad \mu_1 = [1 \ 1]^T$$
$$\varphi_2(\mathbf{x}) = e^{-\|\mathbf{x} - \mu_2\|^2} \quad \mu_2 = [0 \ 0]^T$$

Input $\mathbf{x}$	$\varphi_1(\mathbf{x})$	$\varphi_2(\mathbf{x})$
(1,1)	1	0.1353
(0,1)	0.3678	0.3678
(1,0)	0.3678	0.3678
(0,0)	0.1353	1



The nonlinear φ function transformed a nonlinearly separable problem into a linearly separable one

3.2.5 Comparison with MLP

RBF networks are used to perform complex non-linear pattern classification tasks.

Comparison between RBF networks and multilayer perceptrons:

- Both are examples of *non-linear layered feed-forward* networks.
- Both are *universal approximators*.
- Hidden layers:
 - RBF networks have one *single* hidden layer.
 - MLP networks may have *more* hidden layers.

3.2.5 Comparison with MLP

- Neuron Models:
 - The computation nodes in the hidden layer of a RBF network are *different*. They serve a different purpose from those in the output layer.
 - Typically computation nodes of MLP in a hidden or output layer share a *common neuron model*.
- Linearity:
 - The hidden layer of RBF is *non-linear*, the output layer of RBF is *linear*.
 - Hidden and output layers of MLP are usually *non-linear*.

3.2.5 Comparison with MLP

- Activation functions:
 - The argument of activation function of each hidden unit in a RBF NN computes the *Euclidean distance* between input vector and the center of that unit.
 - The argument of the activation function of each hidden unit in a MLP computes the *inner product* of input vector and the synaptic weight vector of that unit.



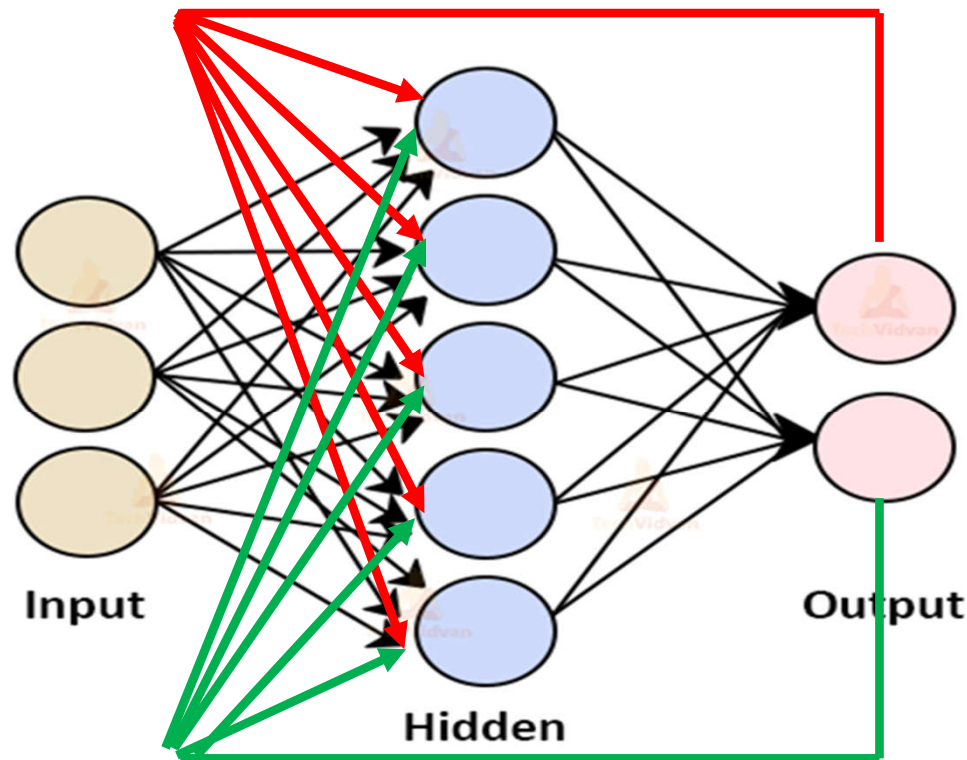
3.3 Recurrent Neural Networks

3.3.1 Introduction

- Recurrent neural networks, also known as RNNs, are a class of neural networks that allow previous outputs to be used as inputs while having hidden states.
- A recurrent neural network (RNN) is one of the two broad types of artificial neural network, characterized by direction of the flow of information between its layers.
- RNNs are bidirectional artificial neural network, meaning that it allows the output from some nodes to affect subsequent input to the same nodes.

3.3 Recurrent neural networks

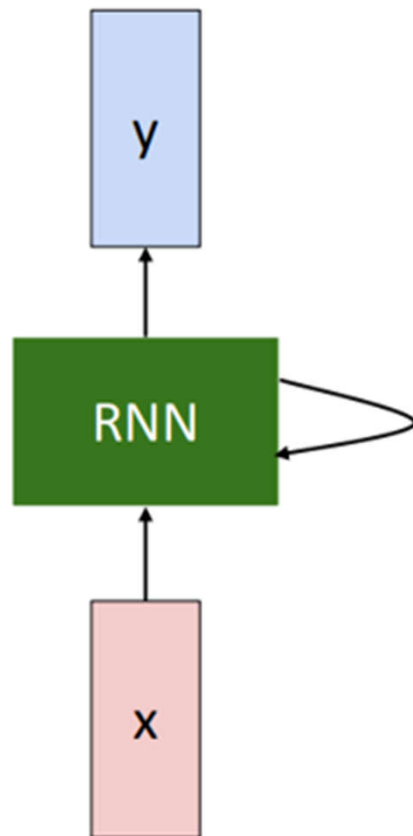
3.3.2 Network Architecture



3.3 Recurrent neural networks

3.3.2 Network Architecture

The simplest recurrent neural network is shown in figure:



x is the input vector of the sequence.

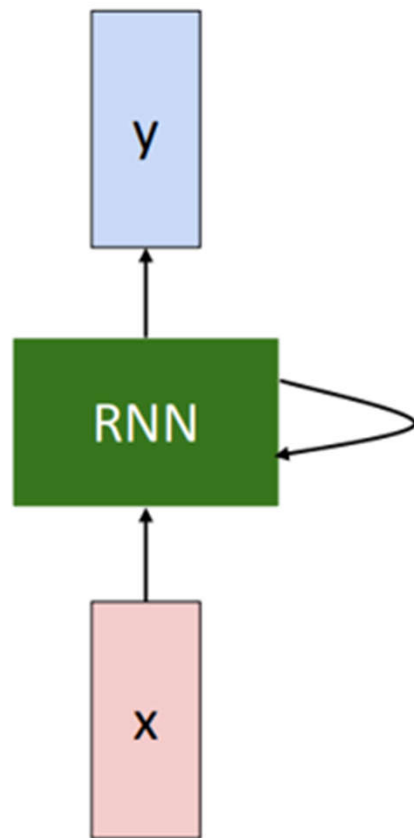
y is the output vector.

A key point here is the presence of the self-loop, which will cause the hidden state of the neural network to change after the input in the sequence.

3.3 Recurrent neural networks

3.3.2 Network Architecture

The simplest recurrent neural network is shown in figure:



The hidden state at time t is given by a function of the input vector at time t and the hidden vector at time $(t - 1)$:

$$h_t = f(h_{t-1}, x_t); \quad y_t = g(h_t) \quad (3.1)$$

x_t is the input vector at time t

h_t is the hidden state at time t

h_{t-1} is the hidden state at time $t-1$

f, g are defined with the use of weight matrices and activation functions.

3.3 Recurrent neural networks

3.3.2 Network Architecture

We can expand Eq.(3.1) and also write the condition for the outputs as follows:

$$h_t = \tanh(W_{hx}x_t + W_{hh}h_{t-1})$$

$$y_t = W_{hy}h_t$$

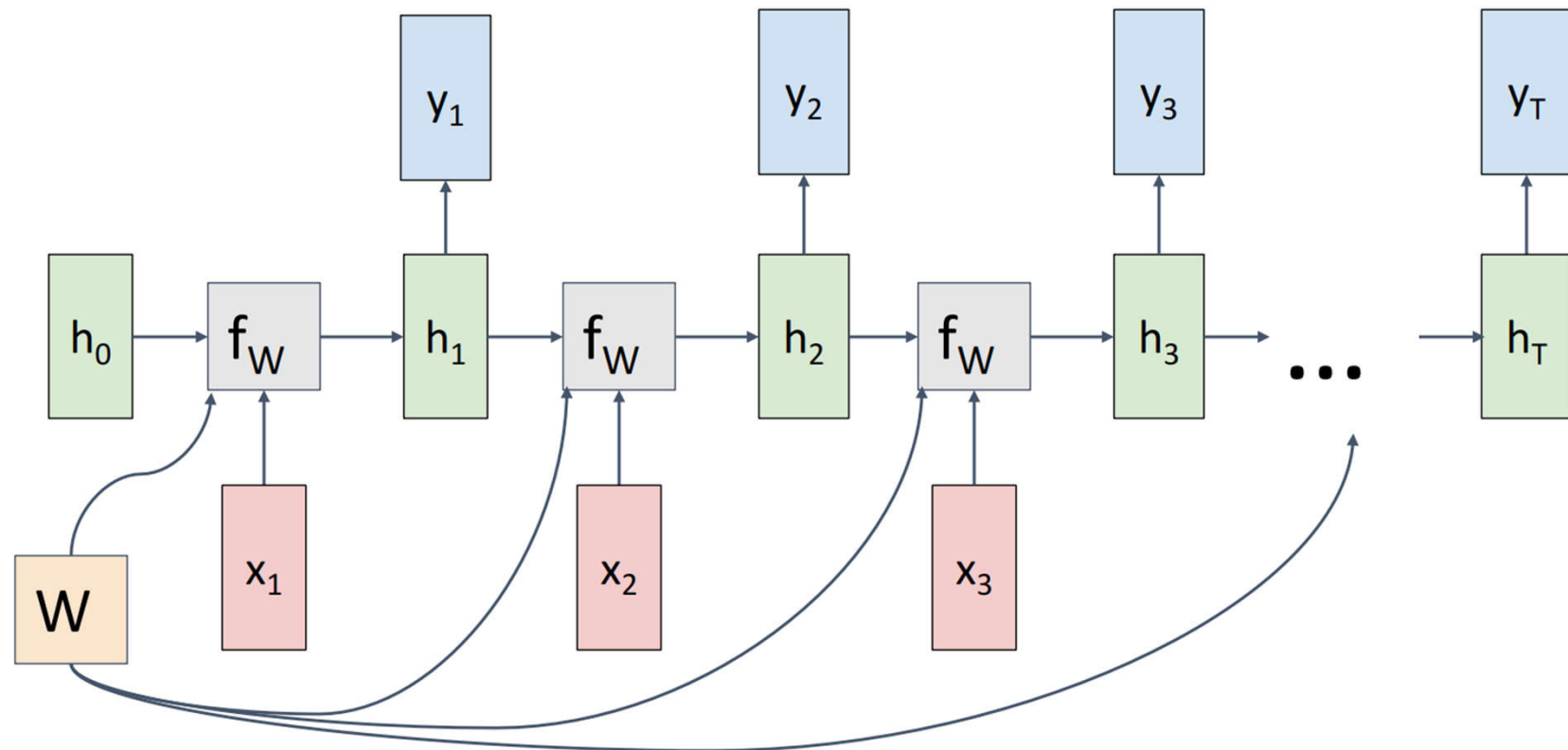
where W_{xh} is a (p $\times$ d) input hidden matrix

W_{hh} is a (p $\times$ p) hidden-hidden matrix

W_{hy} is a (d $\times$ p) hidden-output matrix

3.3 Recurrent neural networks

❖ Example of RNN computational graph:



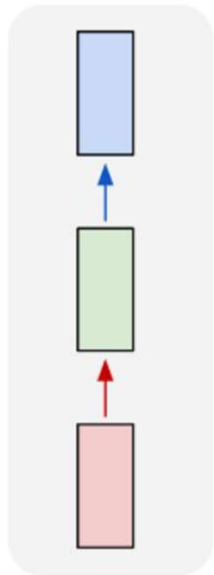
- Re-use the same weight matrix at every time-step

3.3 Recurrent neural networks

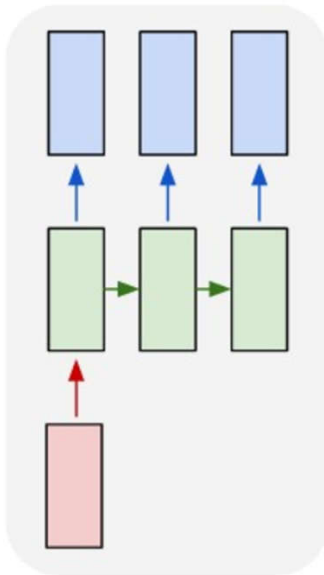
3.3.3 Types of RNN Architecture and application

The different variations of RNNs:

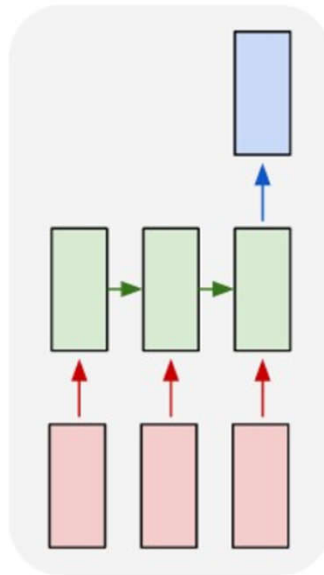
one to one



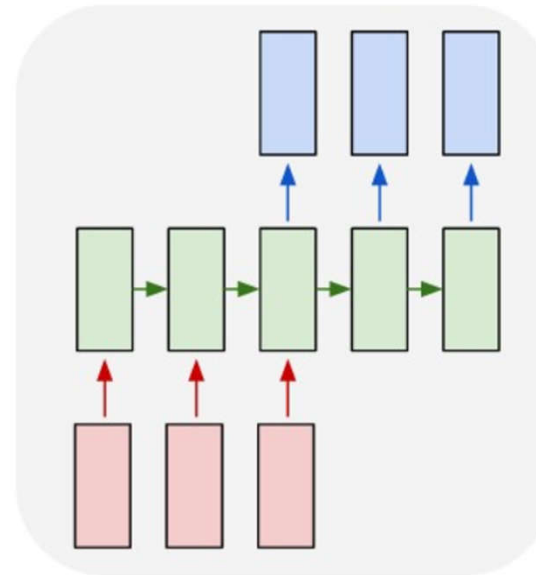
one to many



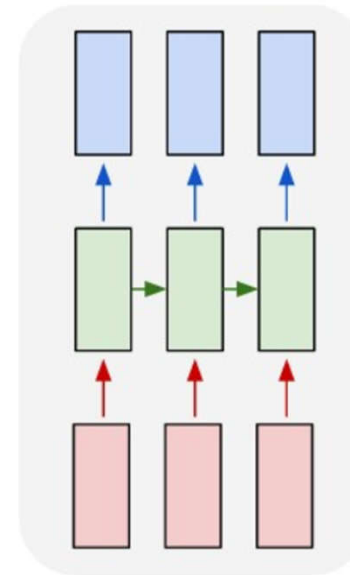
many to one



many to many

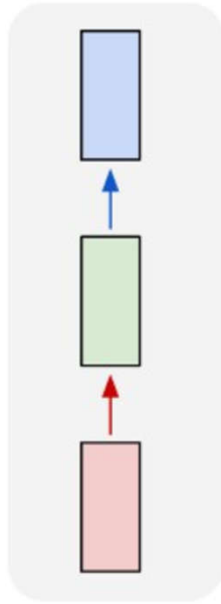


many to many



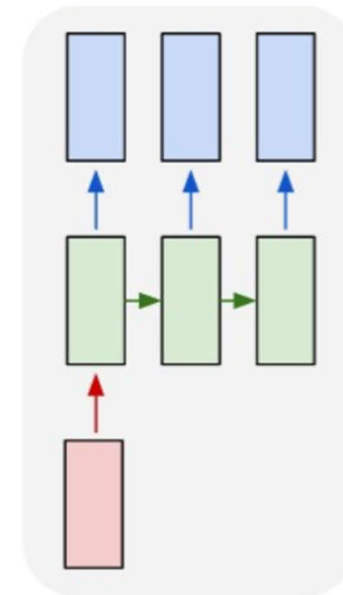
3.3 Recurrent neural networks

one to one



e.g. Image classification:
Image $\rightarrow$ Label

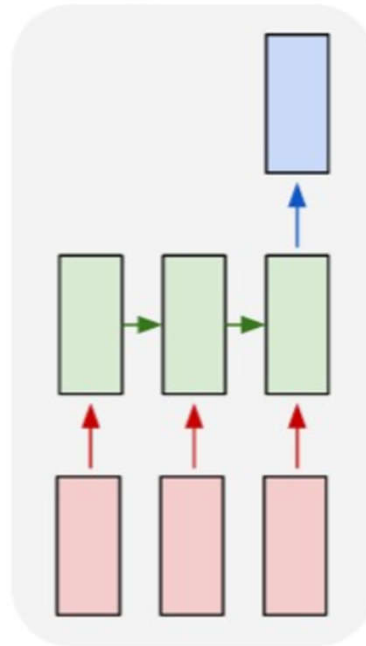
one to many



e.g. Image Captioning:
Image $\rightarrow$ sequence of words

3.3 Recurrent neural networks

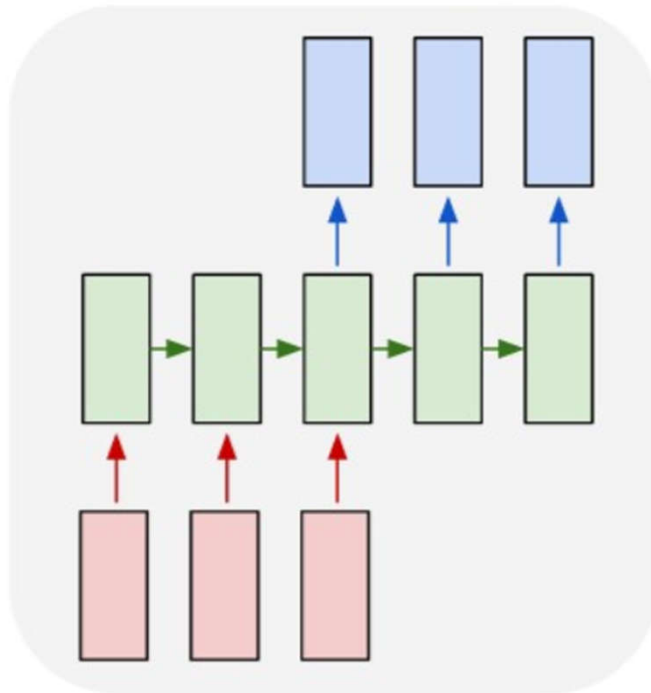
many to one



e.g. Video classification:
Sequence of images $\rightarrow$ Label

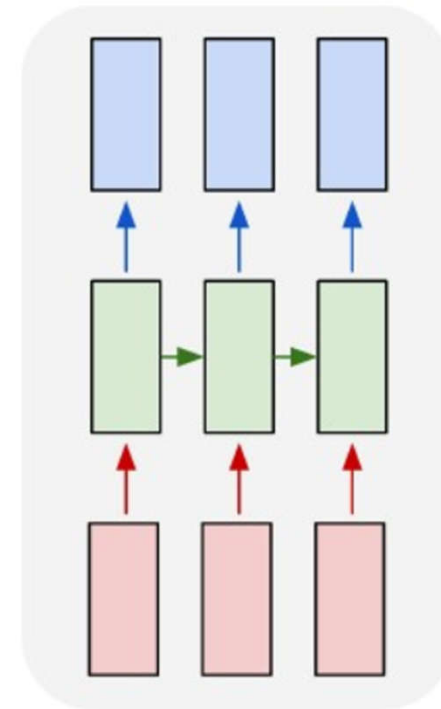
3.3 Recurrent neural networks

many to many



e.g. Machine Translation:
Sequence of words
→ Sequence of words

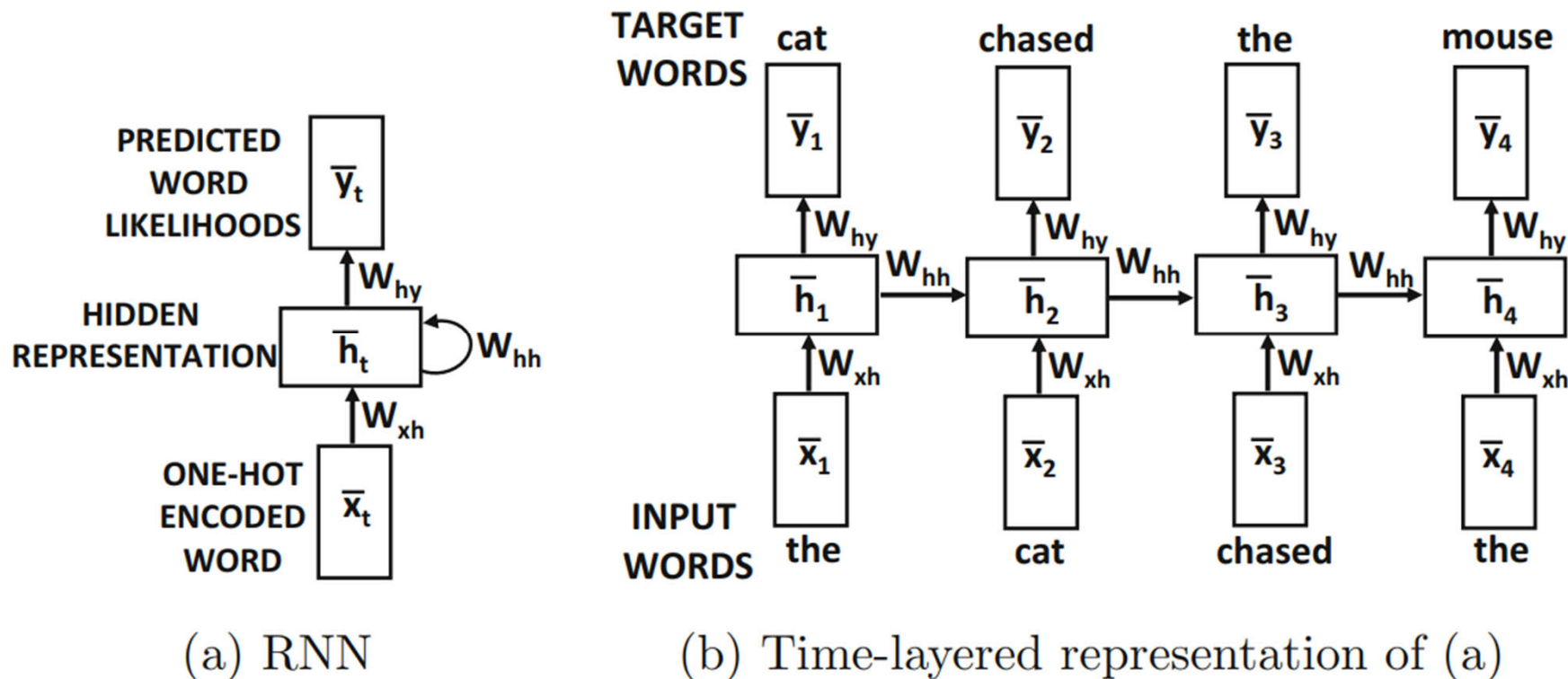
many to many



e.g. Per-frame video classification:
Sequence of images
→ Sequence of labels

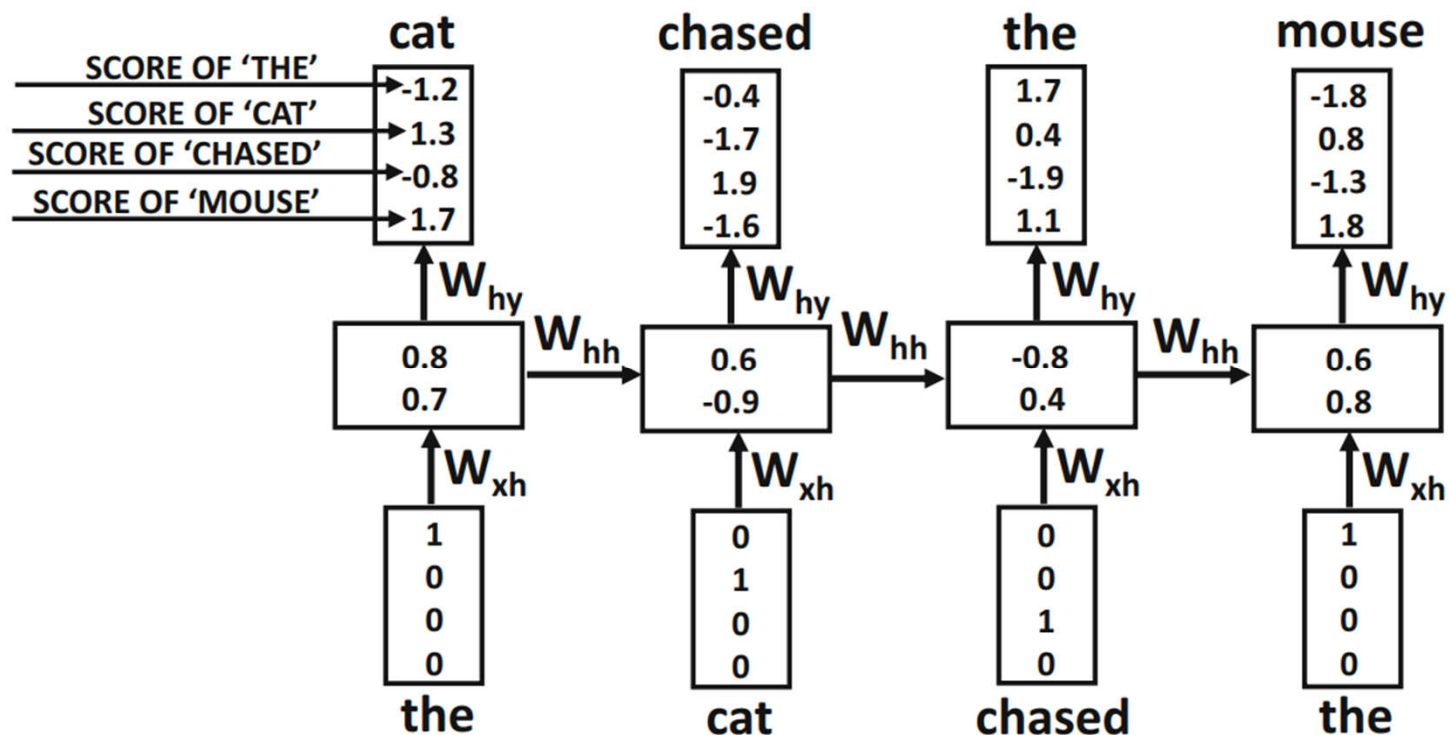
3.3 Recurrent neural networks

- ❖ Example of RNN Language Modeling:
 - Language modeling predicted for the next word in the sequence: “The cat chased the mouse”



3.3 Recurrent neural networks

- ❖ Example of RNN Language Modeling:
 - Language modeling predicted for the next word in the sequence: “The cat chased the mouse”



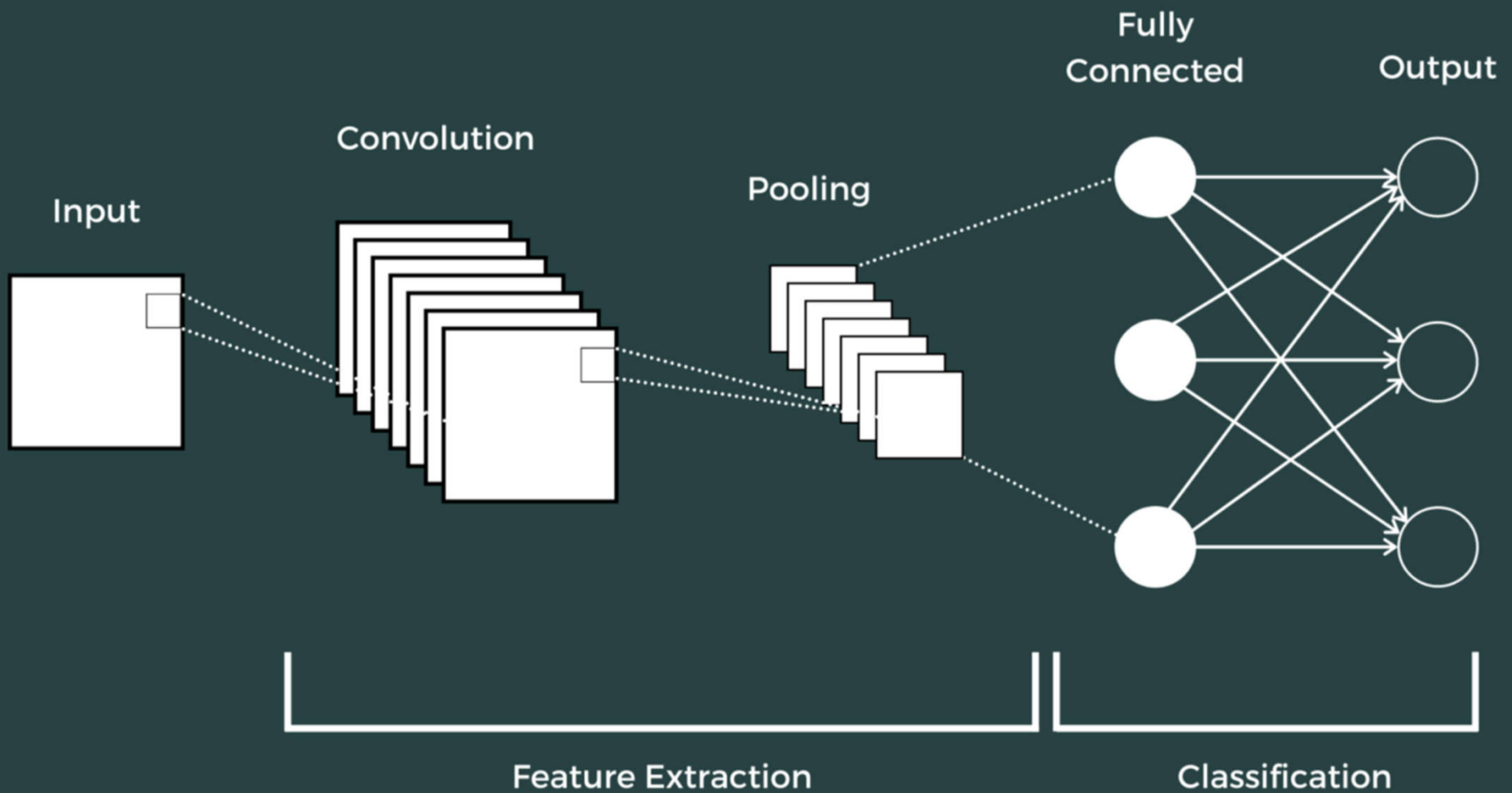


3.4 Convolutional Neural Networks

3.4 Convolutional Neural Network

- Convolutional Neural Network (CNN) is a type of neural network Architecture that is widely used for performing deep learning on image data.
- Similar to Deep Neural Networks (DNNs), CNN also consists of an input layer, a hidden layer(s), and an output layer.
- What differentiates CNNs from ordinary DNNs is that the hidden layers of a CNN consist of a special series of layers called the convolutional and pooling layers.
- CNNs are very effective at recognizing and classifying images, and have been used in a wide range of applications such as image recognition, object detection, and natural language processing

3.4 Convolutional Neural Network



3.4 Convolution Neural Network

- Inputs to feature extraction process are 2-D arrays
- Inputs to fully connected neural networks are vectors.
- The computations performed by both processes consist of:
 - (1) a sum of products is formed
 - (2) a bias value is added.
 - (3) the result is passed through an activation function
 - (4) the activation value becomes a single input to a following layer.

3.4 Convolutional Neural Network

3.4.1 FEATURE EXTRACTION

- Inputs to feature extraction process are 2-D arrays
- Feature extraction process consists of: Convolution layers and pooling layers.

1 Convolution layer

- The set of weights, arranged in the shape with $k \times k$ size (k is odd number), which is called a *kernel*.
- The region of input array that are the same size as the kernel is called *receptive field*
- Convolution layer (sum of products) is performed by moving the kernel over the input array.

3.4 Convolutional Neural Network

➤ Example 3x3 kernel

Kernel

0	-1	0
-1	5	-1
0	-1	0

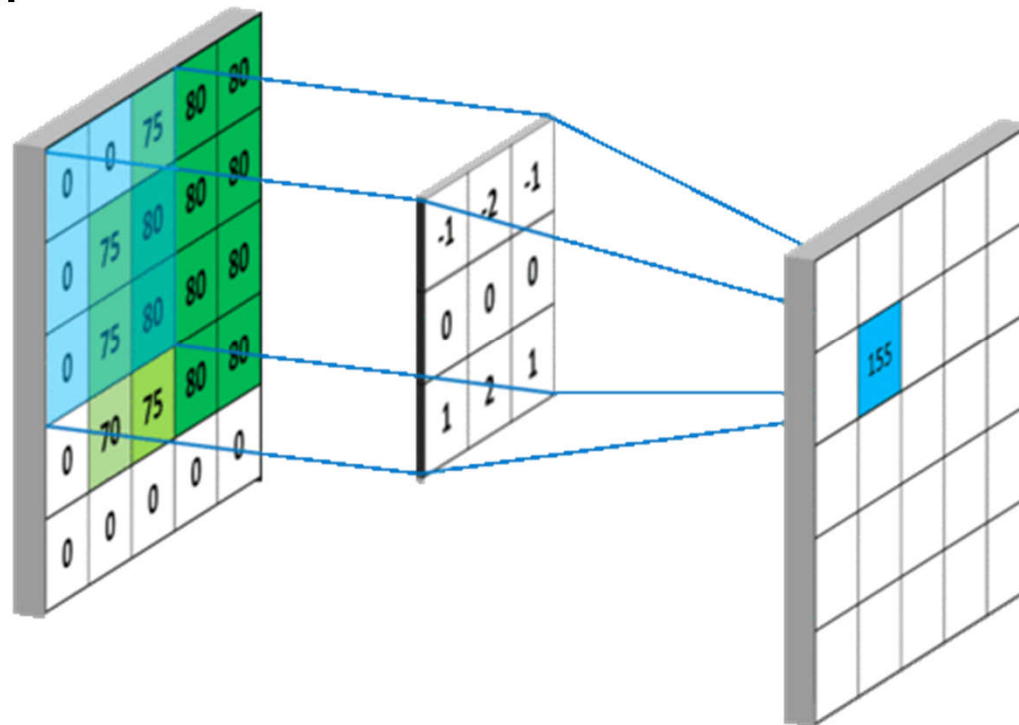
Receptive fields

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

3.4 Convolutional Neural Network

- Convolution is performed by moving the receptive field over the input array and, at each location, forming a sum of products of a set of weights and the values contained in the receptive field.
- The convolution result will produce an array that is smaller size to the original.



The size of result matrix is $(m-k+1) \times (n-k+1)$

3.4 Convolutional Neural Network

- In order for the resulting array to be the same size as the original, we need to perform *padding*.

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

- Convolution with *padding* = p means adding p vectors of 0 to each side of the matrix.

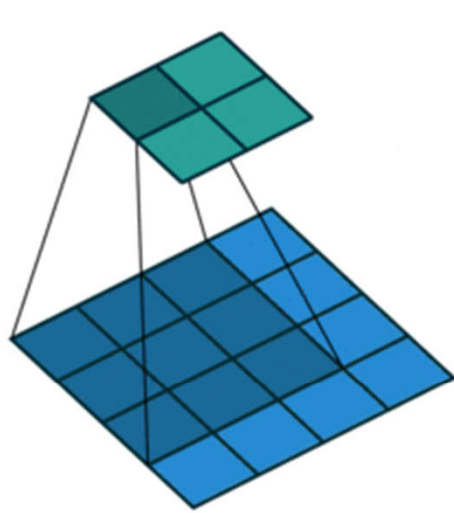
3.4 Convolutional Neural Network

- The number of spatial increments by which a receptive field is moved is called *the stride*.
- In CNNs, using *strides greater than one* for data reduction. For example, using *stride = 2* reduces the input array by one-half in each spatial dimension.
- With *stride = s*, calculate only for values with coordinates: *$l+i*s, l+j*s$*
- In CNNs, with *padding=p*, *stride=s*, kernel size $k \times k$ for array size $m \times n$, the resulting array is

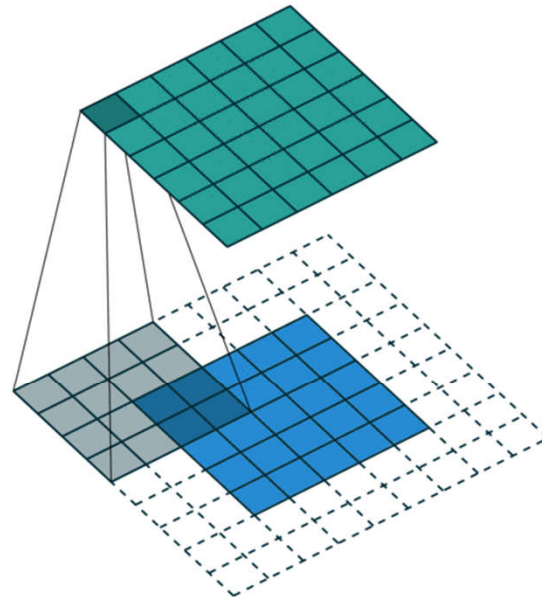
$$\left(\frac{m - k + 2p}{s} + 1 \right) \times \left(\frac{n - k + 2p}{s} + 1 \right)$$

3.4 Convolutional Neural Network

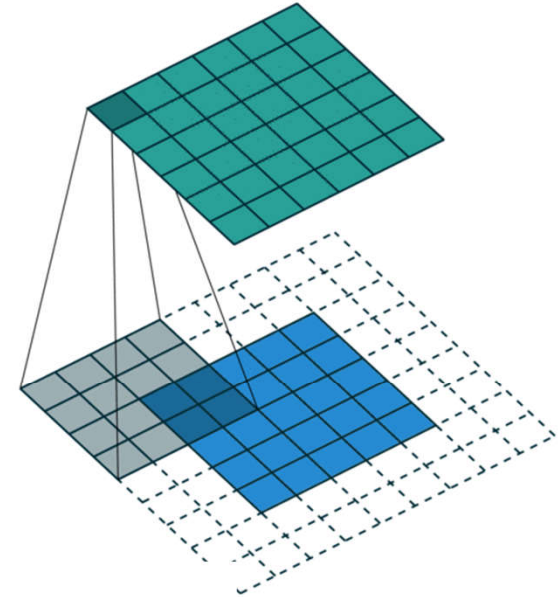
Blue maps are inputs, and cyan maps are outputs.



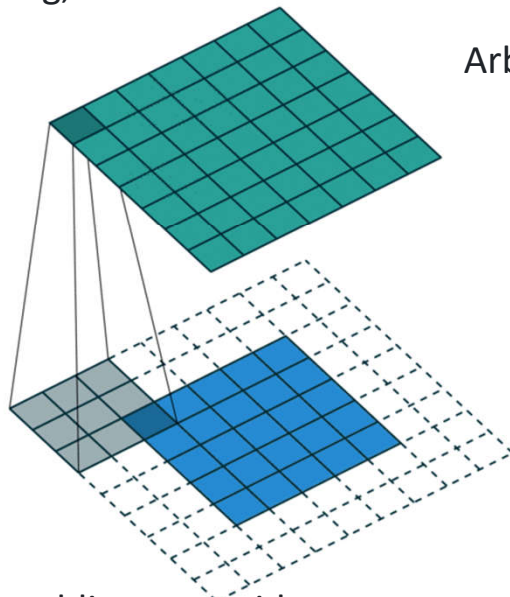
No padding, no strides



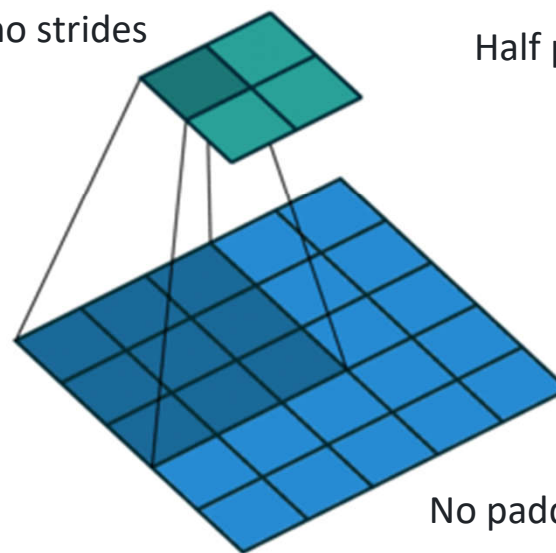
Arbitrary padding, no strides



Half padding, no strides

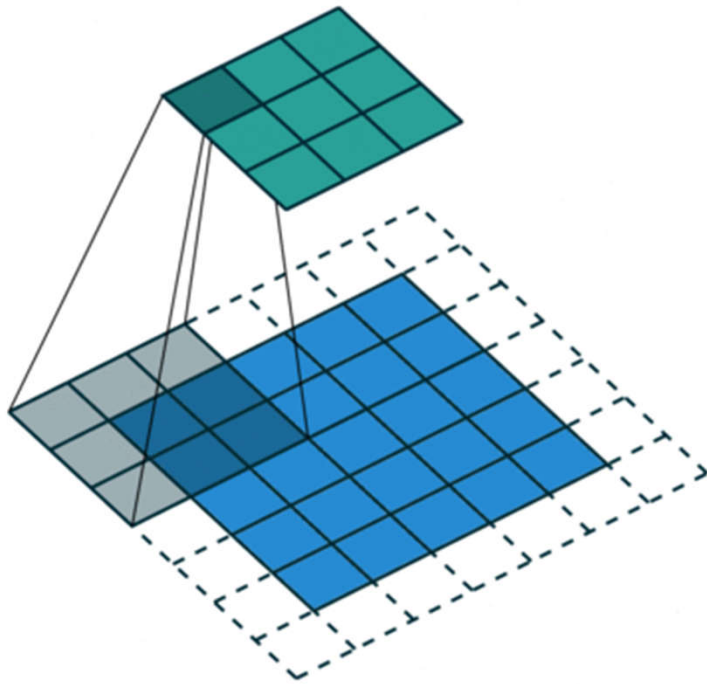


Full padding, no strides

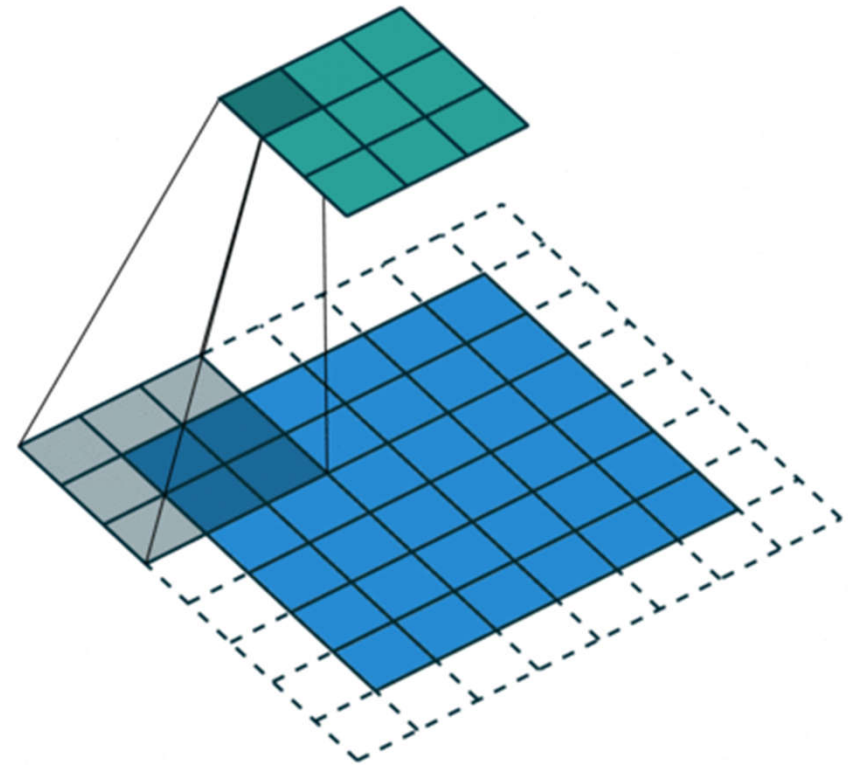


No padding, strides

3.4 Convolutional Neural Network



Padding, strides



Padding, strides (odd)

3.4 Convolutional Neural Network

- To each convolution value (sum of products) we add a bias, then pass the result through an activation function to generate a single value. Then, this value is fed to the corresponding (x, y) location in the input of the next layer. When repeated for all locations in the input array, the process just explained results in a 2-D set of values that store in next layer as a 2-D array, called a *feature map*.
- The same weights and a single bias are used to generate the convolution (*feature map*) values corresponding to all locations of the receptive field in the input array. This is done to cause the same feature to be detected at all points in the array. Using the same weights and bias for this purpose is called *weight* (or *parameter*) *sharing*.

3.4 Convolutional Neural Network

3.4.1 FEATURE EXTRACTION

2 POOLING LAYERS

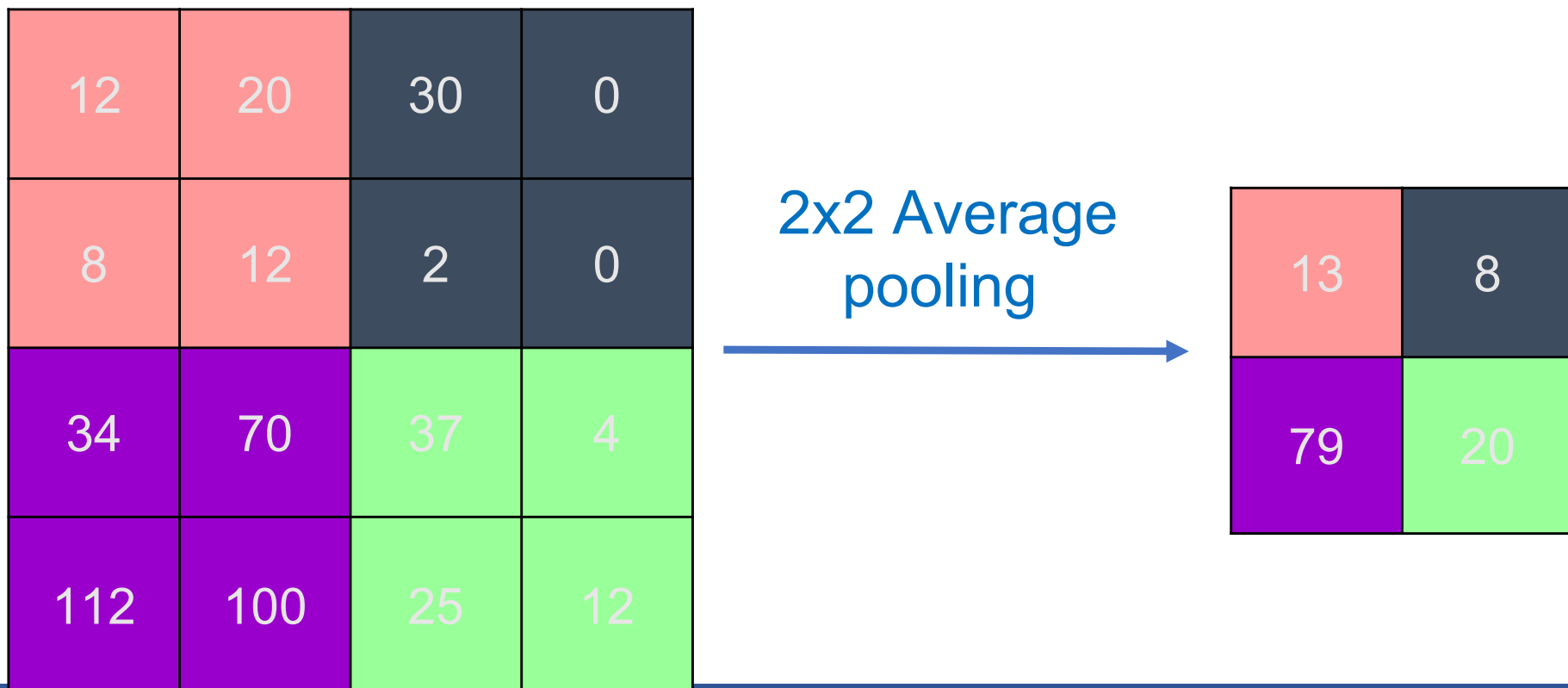
- The process after convolution and activation is *subsampling* (also called *pooling*).
- Pooling is a way of modeling this reduction in dimensionality.
- When training a CNN with large databases, pooling has the additional advantage of reducing the volume of data being processed. The results of subsampling as producing *pooled feature maps*.
- Pooling is done by subdividing a feature map into a set of small (*typically 2x2*) regions, called pooling neighborhoods, and replacing all elements in such a neighborhood by a single value.

3.4 Convolutional Neural Network

2 POOLING LAYERS

Three common pooling methods are:

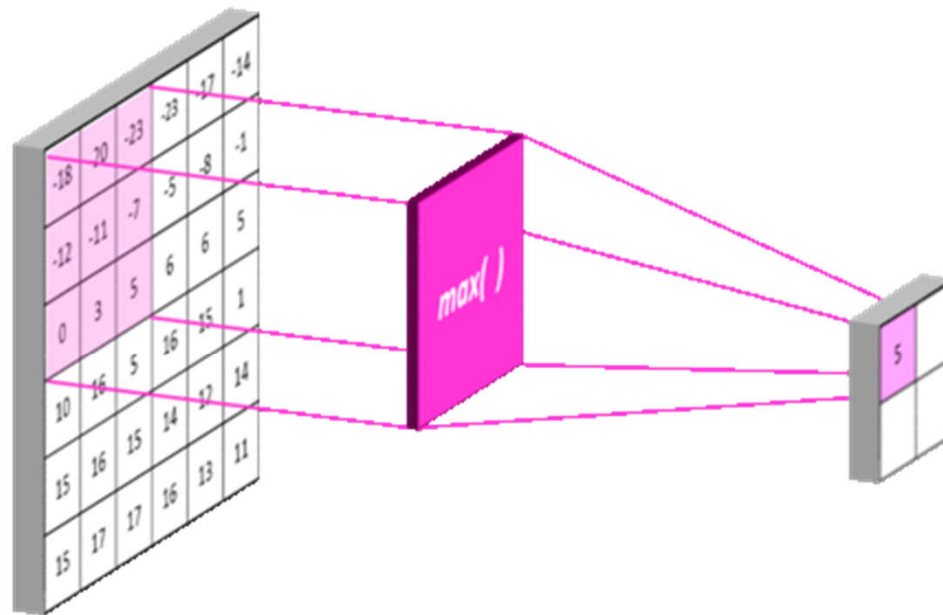
a. **Average pooling**, in which the values in each neighborhood are replaced by the average of the values in the neighborhood.



3.4 Convolutional Neural Network

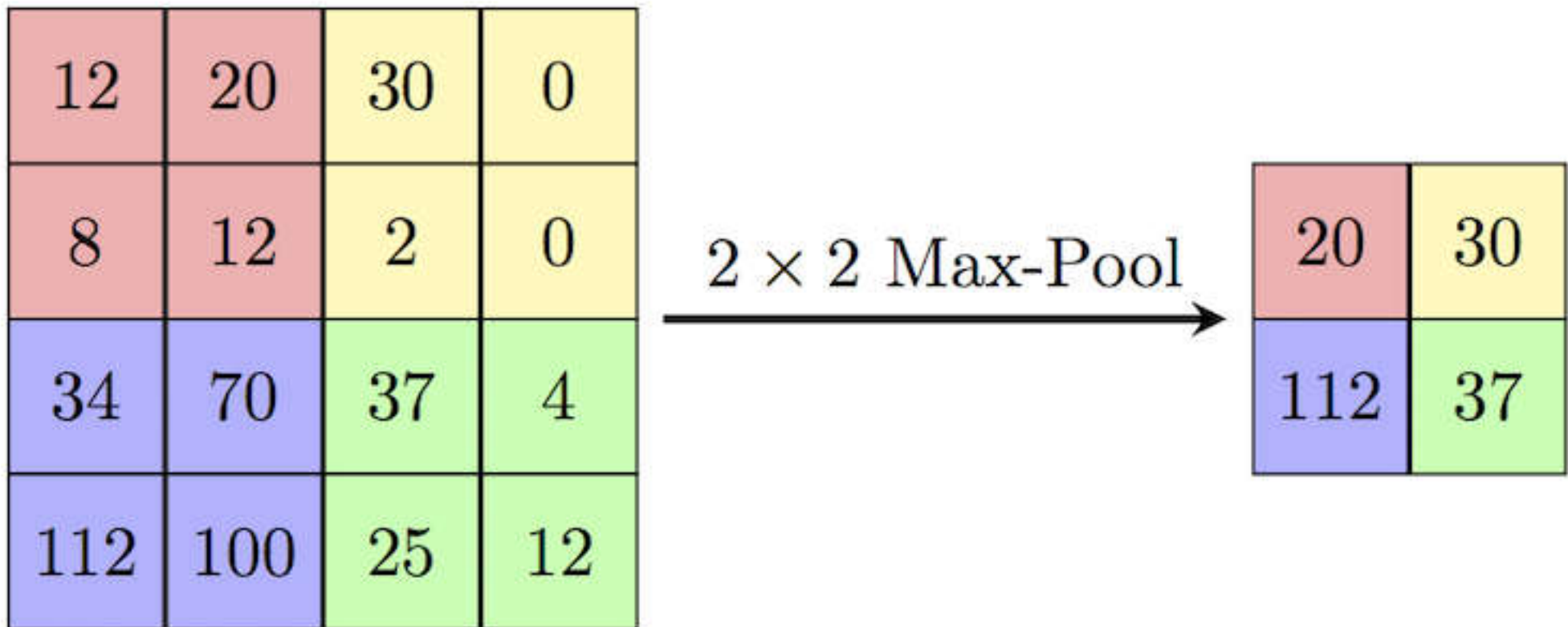
2 POOLING LAYERS

b. **Max-pooling**, which replaces the values in a neighborhood by the maximum value of its elements.



3.4 Convolutional Neural Network

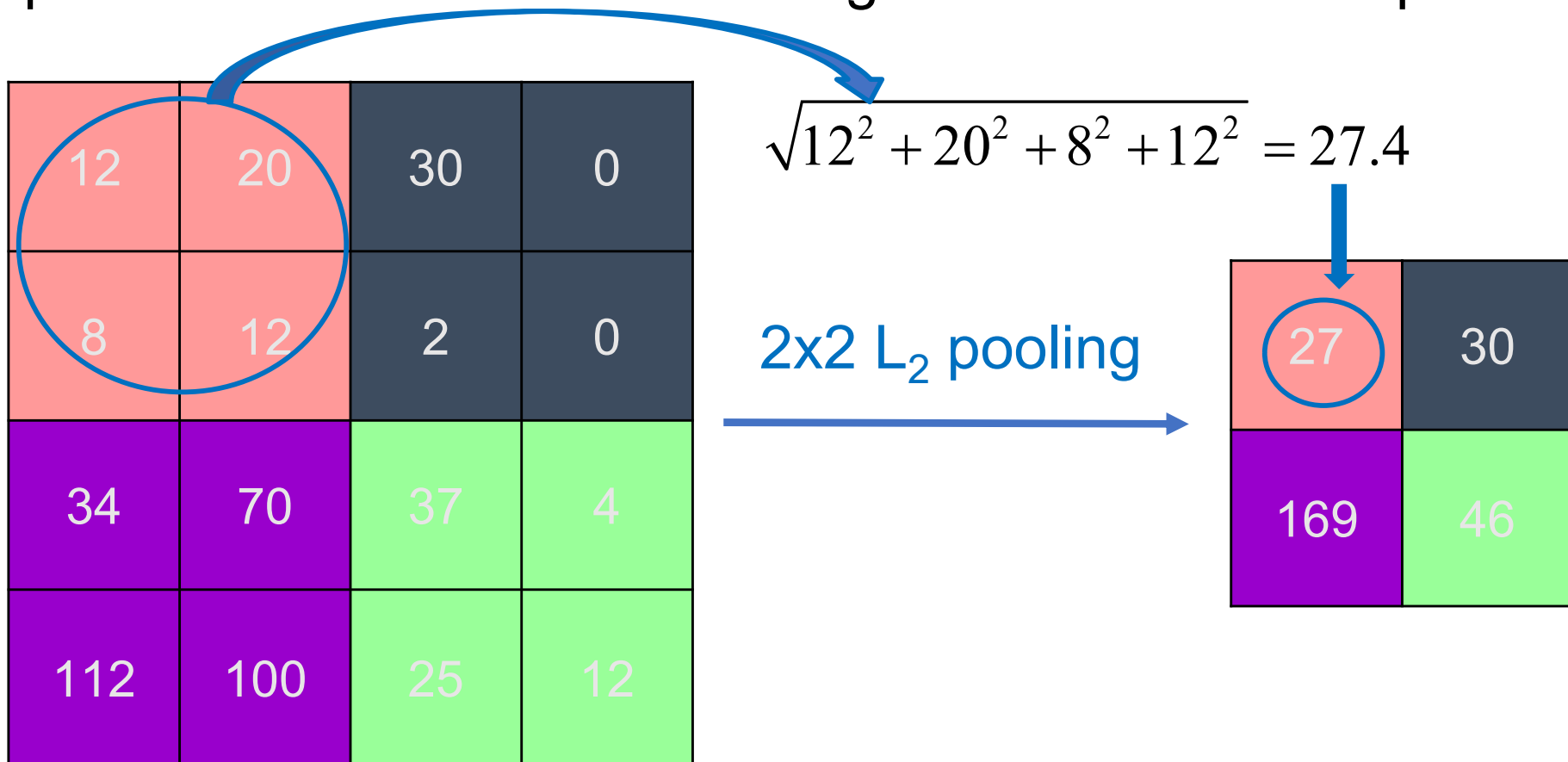
3. POOLING LAYERS



3.4 Convolutional Neural Network

2 POOLING LAYERS

c. L_2 pooling, in which the resulting pooled value is the square root of the sum of the neighborhood values squared.



3.4 Convolutional Neural Network

3.4.2 CLASSIFICATION

- The ultimate objective is to use features for classification, so need a classifier.
- In a CNN we perform classification by feeding the value of the last pooled layer into a fully connected neural net.
- The outputs of a CNN are 2-D arrays, whereas the inputs to a fully connected net are vectors. Therefore, have to *vectorize* the 2-D pooled feature maps in the last layer.
- Each 2-D array in the last layer of the CNN is converted into a vector, then all resulting vectors are concatenated (vertically for a column) to form a single vector.

3.4 Convolutional Neural Network

3.4.2 CLASSIFICATION

- Example *vectorize* the 2-D pooled feature maps in the last layer into column single vector.

12	20	30
8	12	2
112	100	25

*vectorization
(flattening)*



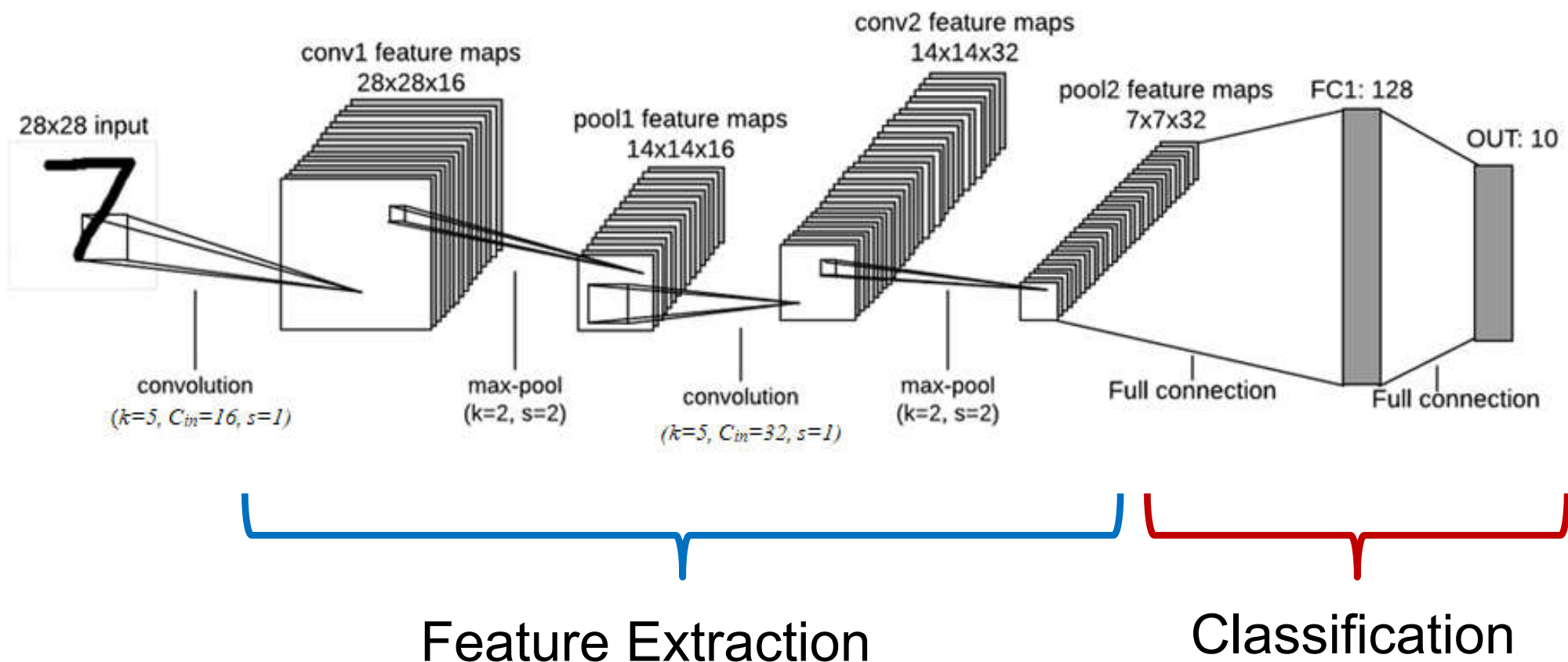
12
20
30
8
12
2
112
100
25

pooled feature maps in the last

3.4 Convolutional Neural Network

Example of CNN

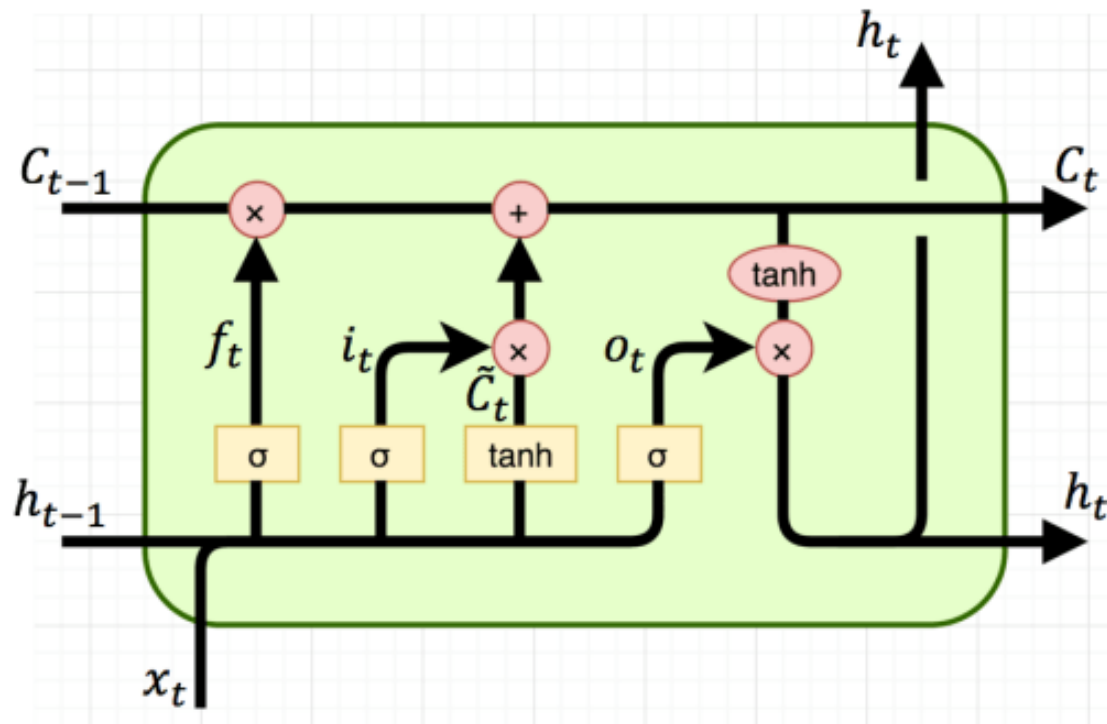
Input image $\rightarrow$ Convolution 1 $\rightarrow$ Pooling 1 $\rightarrow$ Convolution 2 $\rightarrow$ Pooling 2 $\rightarrow$ Full connected $\rightarrow$ Output.

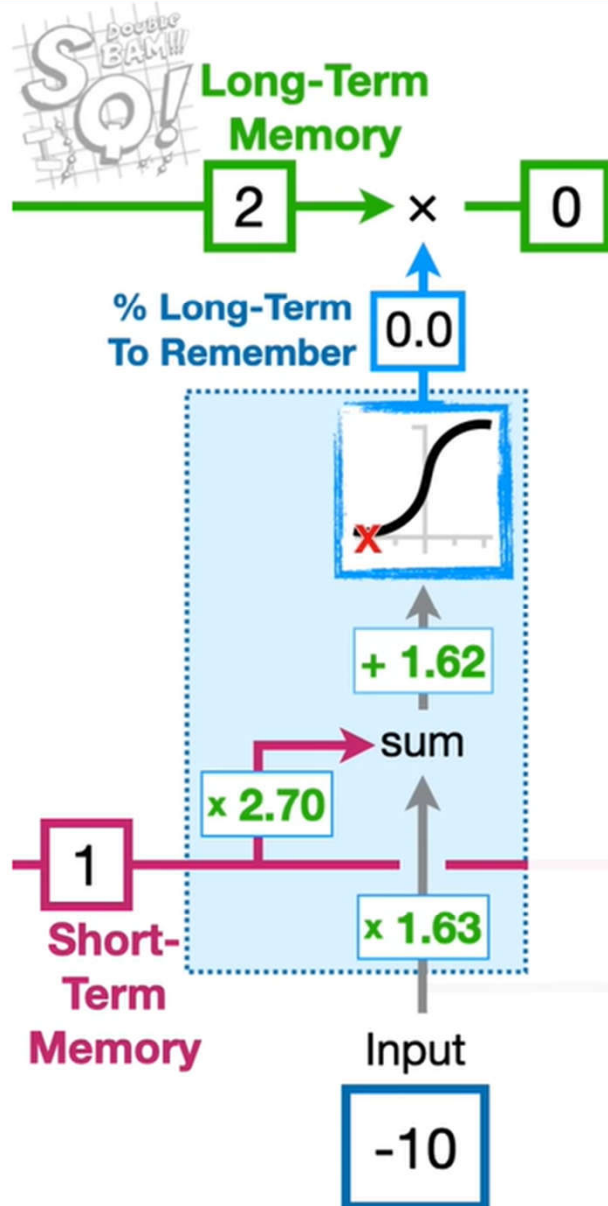




3.5 Long short term memory (LSTM)

LSTM

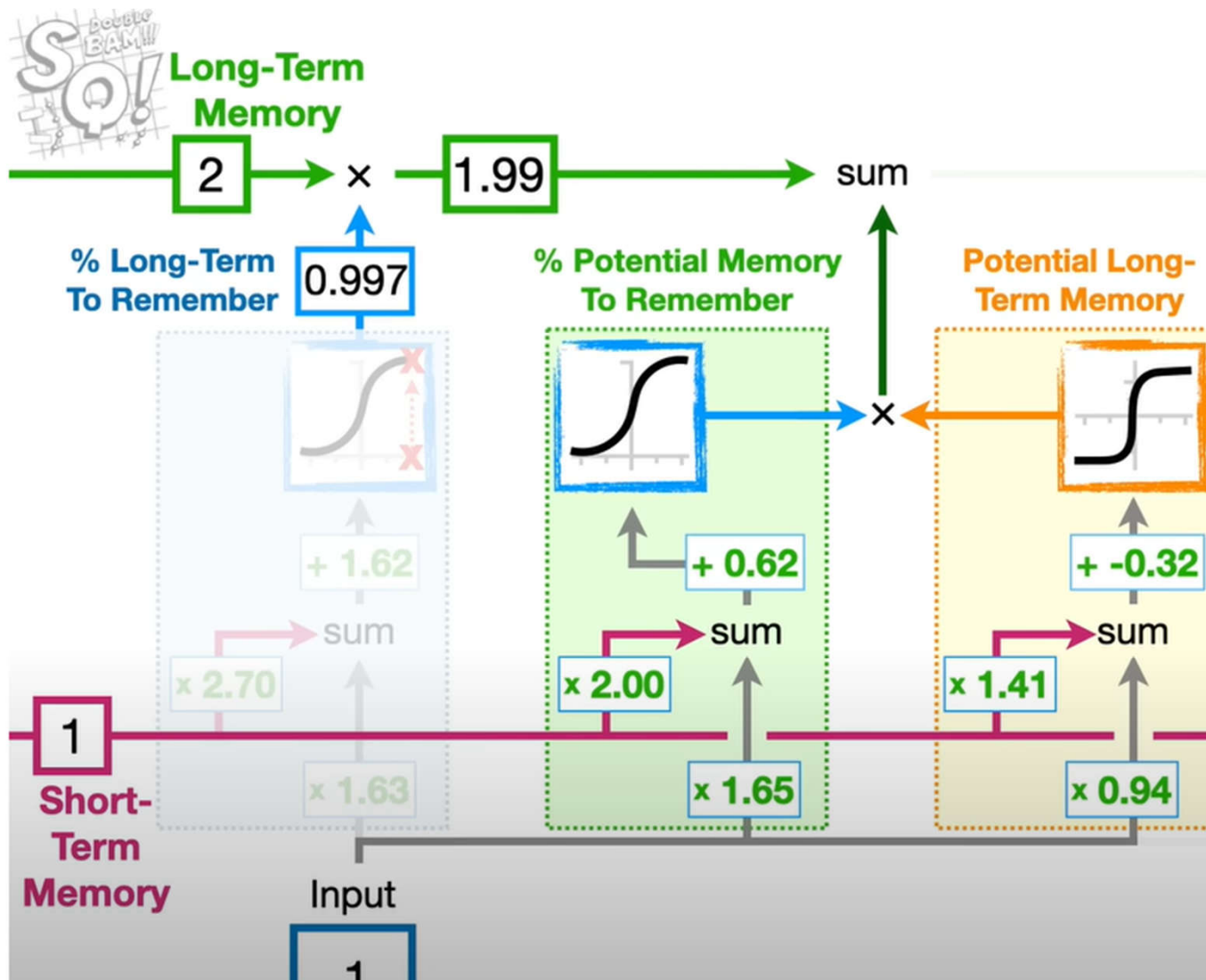




TERMINOLOGY ALERT!!!

Even though this part of the **Long Short-Term Memory** unit determines what percentage of the **Long-Term Memory** will be remembered...

...it is usually called the **Forget Gate**.



So let's plug the numbers in and do the math to see how a **Potential Memory** is created and how much of it is added to the **Long-Term Memory**.

